

RTOS Norms (POSIX)

- Multi-tasking
 - Tasks must be "light-weight".
- Priority scheduling
 - Each task must have priority.
- Pre-emptive
 - High priority task should preempt low priority task (immediately).
 - High priority task should never wait for/due to low priority task.
- Inter-task communication
 - Deterministic latency
- Deterministic memory allocation
- Deterministic interrupt latency
- Deterministic scheduling latency
- Deterministic scheduler duration
- Must handle problem of "priority inversion".

Latencies

- Interrupt latency
 - Time from arrival of interrupt to begin execution of ISR.
 - ARM 7: 24-30 CPU cycles (can vary due to implementation of IRQ_Handler).
 - ARM CM3: 12 CPU cycles (no software IRQ handler)
 - It may delay due to interrupt masking.
- Scheduler latency
 - Time from completion of ISR to begin execution of scheduler.
 - Depends on house-keeping work done by system after ISR call.
- Memory latency
 - Time required for allocating memory block.
 - Memory allocation must be done in fixed time.
- ITC latency
 - Inter-task communication must be completed in fixed time.
 - There can be priority for the messages.

Kernel Response Time

- Time to handle the interrupt, schedule the next process and begin its execution.
 - Interrupt arrived --> Interrupt handler --> ISR call --> ISR execution --> house-keeping -->
 - Scheduler called --> Decide the next process to execute --> Dispatch/restore context of new process.
- Kernel Response Time = Interrupt Latency + ISR Duration + Scheduler Latency + Scheduler Duration
 - Interrupt Latency = Interrupt arrived --> Interrupt handler --> ISR call
 - ISR Duration = ISR execution
 - Scheduler Latency = house-keeping --> Scheduler called
 - Scheduler Duration = Decide the next process to execute

RTOS Performance Metrics

- An RTOS should quickly and predictably respond to the event.
- It should have minimum interrupt latency and fast context switching latency.

Three Models for Performance Measures

(i) Ratio of the sum of latencies of the tasks and Interrupt with respect to the sum of the execution times. (ii) CPU load for how much time CPU not idle (iii) Worst-Case Execution time with respect to mean execution time.

Interrupt latencies

- Interrupt and task execution latencies with respect to the sum of the execution times must be very small
 - sum of task & intr latencies : sum of task execution times
- There must be fast context switching .

CPU Load

- CPU Load = $\sum(T1/P1 + T2/P2 + \dots) + \sum(\text{ISR Duration}) + \sum(\text{RTOS Execution})$
 - cpu load < 100% (to sched all tasks in deadlines)
 - For RMA sched : CPU load < 69.3 % (for m task)
- Each task gives a load to the CPU that equals the task execution time divided by the task period
- CPU load or system load estimation in the case of multitasking is as follows.
 - Suppose there are m tasks. For the multiple tasks, the sum of the CPU loads for all the tasks and ISRs should be less than 1
- CPU load equal to 0.1 (10%)— means the CPU is underutilized and spends its 90% time in a waiting mode.
- Since the executions times can vary or and the task periods vary, the CPU loads can also vary

Worst Case Execution Time

- worst case execution time : average execution time
 - ratio should be close to 1.
 - ie. worst case execution time should not differ much from avg time

Scheduling latency

- House keeping + scheduler execution time + dispatcher(context restore) time
- if system is loaded with multiple tasks, sched latency may differ/ vary

Number of priorities

- more num of unique priorities will have more overhead on system (ie.more memory required and may need more time to select the task)